



# Conversation stages

Make streamed execution progress visible inside a response.

Problem and boundary

Public contract

Working integration

@epam/ai-dial-conversation-stages



# When this library is useful

Make streamed execution progress visible inside a response.

01

## Explain waiting

Show named running stages while content arrives.

02

## Inspect a step

Expand stage  
Markdown and copy  
useful details.

03

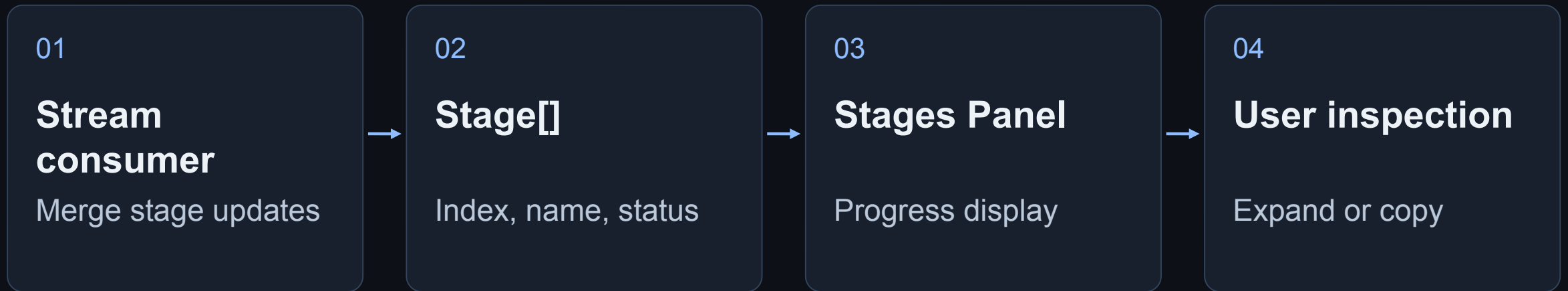
## Reduce noise

Collapse related work  
into a compact group.



# Follow the ownership boundary

Inputs and effects stay explicit.



Library: Stage display, expansion, status icons and Markdown details.



# The public contract to start with

StagesPanel · selected surface; full API index is in the bundle.

| Input or operation | Contract          |
|--------------------|-------------------|
| stages             | Stage[]           |
| isStreaming        | boolean           |
| labels (optional)  | StagesPanelLabels |
| styles (optional)  | StagesPanelStyles |



# A minimal workspace integration

TypeScript / React example · imports and host setup abridged.

```
<StagesPanel
  isStreaming
  stages={[{
    index: 0, name: 'Read demo sources', status: null,
    content: 'Retrieving evidence',
  }]}
  labels={{ runningAriaLabel: 'Running' }}
/>
```

Full example: [examples/conversation-stages.tsx](#) · uses [@epam/ai-dial-conversation-stages](#)



# See a streaming stage in context

Reproducible demo · actual workspace source components

 DEMO · ACTUAL WORKSPACE COMPONENTS

**Inspect an assistant workspace**  
Synthetic data · local source snapshot · no backend calls

**Settings**

**Usage**

Profile

Today

WITHIN LIMITS

**\$3.00** used of \$10.00

\$7.00 left 30%

**Response stages**

☐ Read demo sources >

Retrieving evidence for the demonstration.

## Host-owned stream

The host passes one stage and isStreaming. The panel renders its name, running state and Markdown content.

DEMO · real components rendered from this source snapshot



# Wire a realistic host workflow

Keep conversation stages focused on its public responsibility.

01

## Read stream updates

The host obtains the current stage records.

02

## Supply stage state

Pass stages and `isStreaming` into `StagesPanel`.

03

## Inspect the details

The panel renders stage status and expandable content.



# Available customization

01

## Explicit running state

null status represents a running stage in the shared Stage model.

02

## Streaming awareness

isStreaming controls the active presentation lifecycle.

03

## Grouped detail

CollapsedGroup provides a compact wrapper for related stages.



# Find the integration and its checks

Existing tests are evidence of intended behavior; suite results are not claimed.

| Evidence      | Where to look                                                |
|---------------|--------------------------------------------------------------|
| Real consumer | chat/components/ConversationView/ConversationMessageItem.tsx |
| Test evidence | renders nothing for an empty stage list                      |
| Test evidence | renders all stage rows                                       |



# Constraints that affect integration

01

## Progress is supplied

The component cannot infer the backend work completed.

02

## Keep identity stable

Stage index is used when upstream updates are merged.

03

## Content sensitivity

Only pass execution details intended for the user to see.